



Semaine 2 : Les conditions (if/else)

Objectif : Faire des choix dans ton programme

Durée : 2 heures

Lien Scratch : Le bloc "Si... Alors... Sinon" 



Ce que tu vas apprendre

1. Tester une condition avec `if`
2. Gérer le cas contraire avec `else`
3. Tester plusieurs cas avec `else if`
4. Les opérateurs de comparaison
5. Combiner des conditions avec `&&`, `||`, `!`

1 Le bloc `if` : "Si... Alors..."

En Scratch, tu utilises le bloc "Si... Alors".

En JavaScript, on utilise `if`.

Scratch vs JavaScript

 Si <condition> alors

faire quelque chose

```
if (condition) {  
    // faire quelque chose  
}
```

Exemple simple

```
let age = 12;

if (age >= 10) {
  console.log("Tu peux jouer à ce jeu !");
}
```

Explication :

- `age >= 10` est la **condition** (est-ce que age est supérieur ou égal à 10 ?)
- Si la condition est **vraie** (true), le code entre `{ }` s'exécute
- Si la condition est **fausse** (false), on passe à la suite



Syntaxe importante

```
if (condition) {
  // code à exécuter si la condition est vraie
}
```

- Les parenthèses `( )` autour de la condition sont **obligatoires**
- Les accolades `{ }` délimitent le bloc de code



Exercice : `exercice_1_if_simple.js`

2 Les opérateurs de comparaison

Pour créer des conditions, on compare des valeurs :

Opérateur	Signification	Exemple	Résultat
<code>===</code>	Égal à	<code>5 === 5</code>	<code>true</code>
<code>!==</code>	Différent de	<code>5 !== 3</code>	<code>true</code>
<code>></code>	Supérieur à	<code>10 > 5</code>	<code>true</code>
<code><</code>	Inférieur à	<code>3 < 7</code>	<code>true</code>
<code>>=</code>	Supérieur ou égal	<code>5 >= 5</code>	<code>true</code>
<code><=</code>	Inférieur ou égal	<code>4 <= 3</code>	<code>false</code>

Exemples

```
let score = 100;
let vies = 0;
let pseudo = "Steve";

console.log(score === 100);    // true
console.log(score !== 100);    // false
console.log(vies > 0);         // false
console.log(pseudo === "Steve"); // true
console.log(pseudo === "Alex"); // false
```

⚠ Attention : `===` vs `==`

```
// Toujours utiliser === (triple égal)
console.log(5 === "5");    // false (nombre vs texte)
console.log(5 == "5");     // true (⚠ comparaison "souple", à éviter)
```

Règle : Utilise toujours `===` et `!==` pour éviter les surprises !



Exercice : exercice_2_comparaisons.js

3 Le bloc `if...else` : "Si... Sinon..."

En Scratch, tu utilises le bloc "Si... Alors... Sinon".

Scratch vs JavaScript

Si <condition> alors

faire A

Sinon

faire B

```
if (condition) {  
    // si vrai  
} else {  
    // si faux  
}
```

Exemple

```
let pv = 0;  
  
if (pv > 0) {  
    console.log("Tu es encore en vie !");  
} else {  
    console.log("💀 Game Over !");  
}
```

Exemple Minecraft

```
let blocsDeDiamant = 3;

if (blocsDeDiamant >= 5) {
  console.log("💎 Tu peux crafter une pioche en diamant !");
} else {
  console.log(`Il te manque ${5 - blocsDeDiamant} diamants...`);
}
```

 **Exercice :** exercice_3_if_else.js

4 Le bloc `else if` : plusieurs conditions

Parfois on a plus de 2 choix possibles !

Exemple : Système de notes

```
let note = 15;

if (note >= 16) {
  console.log("🏆 Excellent !");
} else if (note >= 14) {
  console.log("😊 Très bien !");
} else if (note >= 10) {
  console.log("👍 Bien !");
} else {
  console.log("📖 Il faut réviser...");
}
```

Exemple Minecraft : Niveau de difficulté

```
let niveau = 25;

if (niveau >= 50) {
  console.log("🐉 Tu peux affronter l'Ender Dragon !");
} else if (niveau >= 30) {
  console.log("💀 Tu peux aller dans le Nether !");
} else if (niveau >= 10) {
  console.log("🗡️ Tu peux explorer des donjons !");
} else {
  console.log("🏠 Reste près de ta maison pour l'instant...");
}
```

Comment ça marche ?

1. JavaScript teste la **première** condition
2. Si elle est vraie → exécute le bloc et **s'arrête là**
3. Si elle est fausse → teste la condition suivante
4. `else` attrape tous les cas restants

 **Exercice :** `exercice_4_else_if.js`

5 Combiner des conditions : `&&`, `||`, `!`

Parfois une seule condition ne suffit pas !

Opérateur	Nom	Signification
<code>&&</code>	ET (AND)	Les DEUX conditions doivent être vraies
<code> </code>	OU (OR)	AU MOINS UNE condition doit être vraie
<code>!</code>	NON (NOT)	Inverse la condition

L'opérateur `&&` (ET)

```
let age = 12;
let aPermission = true;

if (age >= 10 && aPermission === true) {
  console.log("Tu peux jouer en ligne !");
}
// Il faut avoir 10+ ans ET la permission
```

L'opérateur `||` (OU)

```
let pseudo = "Steve";

if (pseudo === "Steve" || pseudo === "Alex") {
  console.log("Bienvenue, joueur Minecraft !");
}
// C'est OK si c'est Steve OU Alex
```

L'opérateur `!` (NON)

```
let estMort = false;

if (!estMort) {
  console.log("Tu peux continuer à jouer !");
}
// !false = true, donc "si pas mort"
```

Exemple combiné : Accès au Nether

```
let aPortail = true;
let aBriquet = true;
let niveau = 15;

if ((aPortail && aBriquet) && niveau >= 10) {
  console.log("🔥 Tu peux aller dans le Nether !");
} else {
  console.log("Il te manque quelque chose...");
}
```

 Exercice : exercice_5_operateurs_logiques.js

6 Conditions imbriquées

On peut mettre des `if` dans des `if` !

```
let aEpee = true;
let pvMonstre = 50;

if (aEpee) {
  console.log("Tu dégaines ton épée !");

  if (pvMonstre <= 20) {
    console.log("Le monstre est faible, tu l'achèves !");
  } else {
    console.log("Le combat va être rude...");
  }
} else {
  console.log("Tu n'as pas d'arme ! Tu fuis !");
}
```

 Exercice : exercice_6_conditions_imbriquees.js

Résumé

Concept	Syntaxe
Si	<code>if (condition) { }</code>
Sinon	<code>else { }</code>
Sinon si	<code>else if (condition) { }</code>
Égal	<code>===</code>
Différent	<code>!==</code>
Supérieur	<code>> , >=</code>
Inférieur	<code>< , <=</code>
ET	<code>&&</code>
OU	<code> </code>
NON	<code>!</code>

Mini-projet : Choisis ton aventure

À la fin de cette séance, tu sauras créer un mini-jeu avec des choix :

✂ CHOISIS TON AVENTURE ✂

Tu es devant une grotte sombre...

Que fais-tu ?

1. Entrer dans la grotte
2. Contourner la montagne

> 1

Tu entres dans la grotte...

Un ZOMBIE apparaît ! 🧟

Tu as une épée ? OUI

💥 Tu élimines le zombie et trouves un coffre !

➡ Prochaine étape

1. Termine par le défi bonus : `defi_aventure.js`
2. Semaine prochaine : les boucles (for, while) !